

Blockchain-Based Dynamic Group Key Agreement Protocol for Ad Hoc Network

ZHANG Qikun¹, WANG Bingli¹, ZHANG Xiaosong², YUAN Junling¹ and WANG Ruifang¹

(1. Institute of Computer and Communication Engineering, Zhengzhou University of Light Industry,
Zhengzhou 450002, China)

(2. Department of Computer Science and Technology, Tangshan University, Tangshan 063000, China)

Abstract — Group key agreement (GKA) is one of the key technologies for ensuring information exchange security among group members. While GKA is widely used in secure multi-party computation, safety of resources sharing, and distributed collaborative computing. It still has some security flaws and limitations. We propose a Blockchain-based dynamic Group key agreement (BD-GKA) protocol. In contrast to prior works, BD-GKA differs in several significant ways: 1) anonymous identity authentication- it can prevent privacy leaks; 2) traceability-it can track illegal operating entities; 3) load balancing-it balances computation and communication to each node, avoiding the breakdown of single-point and network bottlenecks. This protocol is proven secure under the hardness assumption of decision bilinear Diffie-Hellman. The performance analysis shows that it is more efficient than the referred works.

Key words — Dynamic group key agreement, Blockchain, Anonymous authentication, Information exchange.

I. Introduction

Groups-oriented communication is being used more and more widely, such as multi-party resource sharing video conferencing, telemedicine, multi-party collaborative computing, and so on. The secure communication among group members is very important. As one of the most key technologies to ensure secure communication among group members, Group key agreement (GKA) can be used to consult with a common session key on-line, which is used to encrypt message on open networks by establishing a secure communication channel for group members. Key exchange technology began in 1976 when Diffie and Hellman proposed a two-party

key exchange protocol^[1]. Subsequently, driven by the development of network technique and the change of network application environment, in order to satisfy the application requirements of information technology, three-party key exchange and group key exchange are proposed^[2-4].

A security model for a certificateless group key agreement protocol and a constant-round group key agreement protocol based on certificateless public key cryptography is presented in Ref.[5]. The proposed protocol does not involve any signature scheme, which increases the efficiency of the protocol. A novel one-round authenticated group key agreement protocol is also devised in Ref.[6], the protocol adopts certificateless public key cryptography and abolishes the certificate management burden existing in certificate authority-based public key cryptography and the private-key escrow problem occurring in identity-based cryptography.

A authenticated asymmetric group key agreement protocol is proposed in Ref.[7], and the identity-based public key cryptography is used in these protocols to authenticate users, which can improve the security of group key agreement. A authenticated asymmetric group key agreement based on certificateless cryptosystem is proposed in Ref.[8], which does not require certificates to guarantee the authenticity of public keys yet avoids the inherent escrow problem of identity-based cryptosystems. Later a one-round affiliation-hiding authenticated asymmetric group key agreement with semi-trusted group authority is proposed in Ref.[9], which can protect personal privacy, but the calculation of this protocol is very large, it is not suitable to use in energy-limited mobile devices of wireless network.

A secure and efficient group key agreement protocol is proposed in Ref.[10], which is applicable to cluster-based communication in mobile ad hoc networks. A password-based conditional privacy preserving authentication and group-key generation protocol for VANETs is presented in Ref.[11]. This protocol offers group-key generation, user leaving, user joining, and password change facilities. Privacy-preserving authentication and key agreement protocols are proposed in Ref.[12], which is mainly used for guaranteeing secure and anonymous Device-to-Device group communications.

A flexible asymmetric group key agreement protocol in which information exchange and transmission are orientable is proposed in Ref.[13]. A cross-domain lightweight asymmetric group key agreement to establish a safe and efficient group communication channel between sensor nodes is proposed in Ref.[14]. In this protocol, the computation and communication overhead are lightweight. A dynamic and cross-domain authenticated asymmetric group key agreement is proposed in Ref.[15]. A self-authentication and deniable efficient group key agreement protocol is proposed in Refs.[16,17]. The scheme establishes a group between road side units and vehicles by using self-authentication without certification authority.

For above analysis, the contribution of this protocol is as follows:

1) Non-centralized authentication and privacy preserving. For security, group key agreement requires the participants must to be mutual authenticated before negotiating the group keys to avoid illegal members to participate in group key agreement. This protocol adopts non-centralized authentication. It distributes identity authentication to each node, which is supervised by the entire network node. It uses anonymous identity authentication to avoid disclosure of identity information and protect personal privacy.

2) Distributed computing and communication. For the resource-constrained of mobile terminals, BD-GKA protocol ensures that the computation and communication of each node is as simple and small as possible.

3) Key self-certified. All participants can verify the group key correctness without any other additional communication.

4) Dynamic. BD-GKA protocol supports the members to join or exit the group, and has the group key forward secrecy and backward secrecy security.

II. Basic Knowledge

1. Bilinear mapping

Let G_1 be an additive group and G_2 is a multiplicative cyclic group. Both of the two groups have the same large prime order q , where $q \geq 2^k + 1$, and k is a

security parameter. G_1 and G_2 on the discrete logarithm are difficult, G_1 is generated by g_1 , set $G_1 = \langle g_1 \rangle$. Group G_1 and G_2 are a pair of bilinear group, e is a calculable bilinear mapping, there is $e : G_1 \times G_1 \rightarrow G_2$, and it has the following properties.

- 1) Bilinear: For all $g_1, g_2 \in G_1$, and $a, b \in \mathbb{Z}_q^*$, there is $e(ag_1, bg_2) = e(g_1, g_2)^{ab}$;
- 2) Non-degeneracy: Namely $e(g_1, g_2) \neq 1$.
- 3) Computability: For all $g_1, g_2 \in G_1$, there exists an efficient algorithm to calculate $e(g_1, g_2)$.

2. Computing complexity problems

For given group G_1 is generated by g_1 . For the above definition, the following difficult problems are defined.

Definition 1: Inverse computational Diffe-Hellman (ICDH) problem: The ICDH problem is given g_1, ag_1, abg_1 and for some $a, b \in \mathbb{Z}_q^*$ to compute bg_1 .

Definition 2: Decisional bilinear Diffe-Hellman (DBDH) problem: Suppose the following two triples (abg_1, ag_1, bg_1) and (abg_1, ag_1, π) , any $a, b \in \mathbb{Z}_q^*$ and $\pi \in G_1$ are computationally indistinguishable.

III. The Process of BD-GKA Protocol

1. Initialization

In this section, we describe a lightweight symmetric group key agreement protocol based on blockchain. The protocol consists of KGC (KeyGeneration Center) and network terminals. KGC is a key entity who generates system parameters and master keys. Especially, KGC is mainly used to verify the identity of the terminal entity for tracking the identity of anonymous members in the blockchain.

In this work, it is assumed that the protocol contains a KGC and n terminals. Let $U = \{u_1, u_2, \dots, u_n\}$ be the set of terminals. And the corresponding identity set is $ID = \{id_{u_1}, id_{u_2}, \dots, id_{u_n}\}$.

Initial: input a security parameter ℓ , the KGC chooses an additive group G_1 and a multiplicative group G_2 , the discrete logarithm over G_1 and G_2 is difficult, $g_1 \in G_1$ is a generator of G_1 , G_1 and G_2 have the same large prime number q , and there exists a bilinear map e and $e : G_1 \times G_1 \rightarrow G_2$, $H_2 : G_1 \rightarrow \mathbb{Z}_q^*$ and $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ are two hash functions.

$Setup(1^\lambda) \rightarrow (PK_A, SK_A)$: The $Setup(1^\lambda)$ algorithm is run by KGC, It takes the security parameters λ as input, and outputs the system master key $SK_A \in \mathbb{Z}_q^*$ and public key $PK_A = g_1 SK_A$.

$KeyGen(SK_A, id_u) \rightarrow (sk_u, pk_u)$: The key generation algorithm is run by KGC. It takes the system master key SK_A and a identify id_{u_i} of the terminal u_i ($1 \leq i \leq n$) as input, and outputs the public/ private key pair (sk_{u_i}, pk_{u_i}) of the terminal as follows:

- 1) Calculates $s_{u_i} = H_1(id_{u_i})$.
- 2) Output the private key $sk_{u_i} = SK_A s_{u_i}$ and the

public key $pk_{u_i} = g_1 sk_{u_i}$. The system parameters are $params = (PK_A, q, G_1, G_2, g_1, e, H_1, H_2)$.

2. Computation of group key

The Computation of group key proposed in this protocol is depicted as follows:

1) Each terminal u_i ($1 \leq i \leq n$) participating in the group key agreement broadcasts its IP address and public key to other members.

2) After received all the messages broadcast from other members, each member will arrange the order and number of these IP addresses according to the size of the IP address. Then they save the IP address and the public key of the one before them, as well as keep the IP address and the public key of the one after them. We assume that this ordered set is $Ord = \{(IP_1, pk_{u_1}), (IP_2, pk_{u_2}), \dots, (IP_n, pk_{u_n})\}$.

3) u_1 selects a random number $m_1 \in \mathbb{Z}_q^*$, and calculates $M_1 = m_1 g_1$, $h_{1,1} = H_2(M_1)$, $h_{1,2} = H_2(pk_{u_1}, M_1, sk_{u_1})g_1$ and $Sig_1 = sk_{u_1} h_{1,2} + sk_{u_1} M_1$, where $time_1$ is a timestamp. Then, u_1 encapsulates the messages $\{h_{1,1}, IP_2, h_{1,2}, (M_1), pk_{u_1}, Sig_1\}$ into a block as transaction and sends the transaction to members u_2 whose IP address is IP_2 in the network.

4) After the member u_2 received the messages $\{h_{1,1}, IP_2, h_{1,2}, pk_{u_1}, (M_1), Sig_1\}$ from u_1 , it compares its IP address with the IP address in the message block. If the two IP addresses are equal, u_2 will write a new block. u_2 verifies the identity of u_1 by the equation $e(Sig_1, g_1) = e(h_{1,2}, pk_{u_1})e(M_1, pk_{u_1})$. If it holds, u_2 can ensure the transaction sent by previous block. u_2 selects a random number $m_2 \in \mathbb{Z}_q^*$, calculates $M_2 = m_2 g_1$, $h_{2,2} = H_2(pk_{u_2}, M_2, sk_{u_2})g_1$ and $Sig_2 = sk_{u_2} h_{2,2} + sk_{u_2} M_2$. According to timestamp $time_2$, u_2 computes $h_{2,1} = H_1(h_{1,1}, H_2(h_{1,2}), H_2(Sig_1), time_2, m_2)$ as block header. Then, u_2 encapsulates the messages $\{h_{2,1}, IP_{2+1}, h_{2,2}, pk_{u_2}, (M_1, M_2), Sig_2\}$ into a block as transaction and broadcasts the transaction to the next member in the network.

5) Follow the above processes to execute, after the member u_i ($3 \leq i < n, i \neq 1$) received the messages $\{h_{i-1,1}, IP_i, h_{i-1,2}, pk_{u_{i-1}}, (M_1, \dots, M_{i-1}), Sig_i\}$ from u_{i-1} , it compares its IP address with the IP address in the message block. If the two IP addresses are equal, u_i will write a new block. u_i verifies the identity of u_{i-1} by the equation $e(Sig_{i-1}, g_1) = e(h_{i-1,2}, pk_{u_{i-1}})e(M_{i-1}, pk_{u_{i-1}})$. If it holds, u_i can ensure the transaction sent by previous block. u_i selects a random number $m_i \in \mathbb{Z}_q^*$, calculates $M_i = m_i g_1$, $h_{i,2} = H_2(pk_{u_i}, M_i, sk_{u_i})g_1$ and $Sig_i = sk_{u_i} h_{i,2} + sk_{u_i} M_i$. According to timestamp $time_i$, u_i computes $h_{i,1} = H_1(h_{i-1,1}, H_2(h_{i-1,2}), H_2(Sig_{i-1}), time_i, m_i)$ as block header. Then, u_i encapsulates the messages $\{h_{i,1}, IP_{i+1}, h_{i,2}, pk_{u_i}, (M_1, M_2, \dots, M_i), Sig_i\}$ into a

block as transaction and broadcasts the transaction to next member in the network. The block is shown in Fig.1.

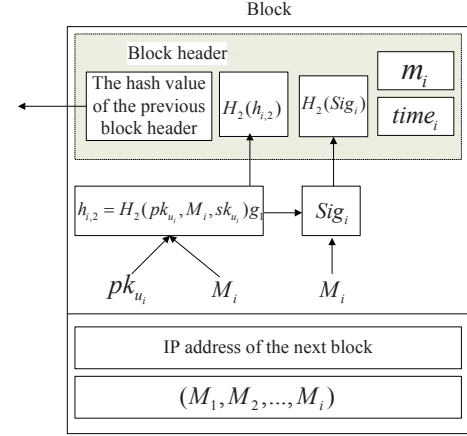


Fig. 1. Block structure of blockchain

6) After receiving the transaction broadcast from u_i , the member u_{i+1} ($i < n$) compares its IP address with the IP address in this transaction. When u_{i+1} finds that its IP address is the same as the IP address in the transaction, it writes a new block, and it does the work as same as u_i .

7) When the last member u_n received the transaction $\{h_{n-1,1}, IP_n, h_{n-1,2}, pk_{u_{n-1}}, (M_1, \dots, M_{n-1}), Sig_{n-1}\}$ broadcast from u_{n-1} , it compares its IP address with the IP address in the message block. If the two IP addresses are equal, u_n will write a new block. u_n verifies the identity of u_{n-1} by the equation $e(Sig_{n-1}, g_1) = e(h_{n-1,2}, pk_{u_{n-1}})e(M_{n-1}, pk_{u_{n-1}})$. If it holds, u_n can ensure the transaction sent by previous block. u_n selects two random numbers $t_n, m_n \in \mathbb{Z}_q^*$, calculates $M_n = m_n g_1$, $f_1 = t_n M_1, f_2 = t_n M_2, \dots, f_n = t_n M_n$, $h_{n,2} = H_2(pk_{u_n}, M_n, sk_{u_n})g_1$, $\beta_n = g_1 \sum_{r=1}^n f_r$ and $Sig_n = sk_{u_n} h_{n,2} + \beta_n g_1$. According to timestamp $time_n$, u_n computes $h_{n,1} = H_1(h_{n-1,1}, H_2(h_{n-1,2}), H_2(Sig_{n-1}), time_n, m_n)$ as block header. Then, u_n encapsulates the messages $\{h_{n,1}, null, h_{n,2}, pk_{u_n}, (M_1, M_2, \dots, M_n), Sig_n\}$ into a block. u_n can calculate the group key $gk_{u_n} = e(t_n g_1, \beta_n g_1)$, and broadcast the messages $\{h_{n,1}, null, h_{n,2}, pk_{u_n}, (f_1, f_2, \dots, f_n), Sig_n\}$ to all the members in the network.

8) Each member u_k ($1 \leq k \leq n-1$) received the messages $\{h_{n,1}, null, h_{n,2}, pk_{u_n}, (f_1, f_2, \dots, f_n), Sig_n\}$ from u_n . They verify the identity of u_n by the equation $e(Sig_n, g_1) = e(h_{n,2}, pk_{u_n})e(g_1 \sum_{r=1}^n f_r, pk_{u_n})$. If it holds, u_k can ensure the transaction sent by u_n . u_k computes $\alpha_k = m_k^{-1} f_k = t_n g_1$, $\beta_k = \sum_{\tau=1}^n f_\tau = t_n \sum_{\tau=1}^n M_\tau$ and the group key $gk_{u_k} = e(\alpha_k, \beta_k)$.

9) Group keys self-certified. All the group members $u_l (1 \leq l \leq n)$ have calculated group keys, they do not need to broadcast the hash value of the group key to other group members and verify that the group key is correct by comparing the hash values of the group keys broadcast by other members. They can verify whether the equation $e(\alpha_l, \sum_{i=1}^n M_i) = e(\beta_l, g_1)$ holds to check the correctness of group keys they computed.

3. Group key update for nodes join group

When a new node wants to join the existing group, the join phase occurs in this case. The existing group should provide a fair group key formation to the new node. However, it should ensure that none of new members can calculate any previous group decryption key. Suppose the existing group member set is $U = \{u_1, u_2, \dots, u_n\}$, the corresponding identity set is $ID = \{id_{u_1}, id_{u_2}, \dots, id_{u_n}\}$, the new terminal node u_{n+1} wishes to join this group, and the public/private key pair of the terminal u_{n+1} is $(sk_{u_{n+1}}, pk_{u_{n+1}})$. The new set of terminal node is $U' = U \cup \{u_{n+1}\} = \{u_1, u_2, \dots, u_{n+1}\}$. The updated steps of group key are as follows:

1) u_{n+1} broadcasts its IP address and public key messages $(IP_{n+1}, pk_{u_{n+1}})$ to the group in the network.

2) After receiving the messages from u_{n+1} , u_n computes $Sig'_n = sk_{u_n} h_{n,2} + sk_{u_n} M_n$ and encapsulates the messages $\{h_{n,1}, IP_{n+1}, h_{n,2}, pk_{u_n}, (M_1, M_2, \dots, M_n), Sig'_n\}$ into a block as transaction and broadcasts the transaction to u_{n+1} .

3) When the member u_{n+1} received the transaction $\{h_{n,1}, IP_{n+1}, h_{n,2}, pk_{u_n}, (M_1, M_2, \dots, M_n), Sig'_n\}$ broadcast from u_n , it compares its IP address with the IP address in the message block. If the two IP addresses are equal, u_{n+1} will write a new block. u_{n+1} verifies the identity of u_n by the equation $e(Sig'_n, g_1) = e(h_{n,2}, pk_{u_n})e(M_n, pk_{u_n})$. If it holds, u_{n+1} can ensure the transaction sent by previous block. u_{n+1} selects two random numbers $t_{n+1}, m_{n+1} \in \mathbb{Z}_q^*$, calculates $M_{n+1} = m_{n+1}g_1, f'_1 = t_{n+1}M_1, f'_2 = t_{n+1}M_2, \dots, f'_{n+1} = t_{n+1}M_{n+1}, h_{n+1,2} = H_2(pk_{u_{n+1}}, M_{n+1}, sk_{u_{n+1}})g_1, \beta_{n+1} = g_1 \sum_{r=1}^{n+1} f_r$ and $Sig_{n+1} = sk_{u_{n+1}} h_{n+1,2} + \beta_{n+1}g_1$. According to timestamp $time_{n+1}$, u_{n+1} computes $h_{n+1,1} = H_1(h_{n,1}, H_2(h_{n,2}), H_2(Sig'_n), time_{n+1}, m_{n+1})$ as block header. Then, u_{n+1} encapsulates the messages $\{h_{n+1,1}, null, h_{n+1,2}, pk_{u_{n+1}}, (M_1, M_2, \dots, M_{n+1}), Sig_{n+1}\}$ into a block. u_{n+1} can calculate the group key $gk_{u_{n+1}} = e(t_{n+1}g_1, \beta_{n+1}g_1)$, and broadcast the messages $\{h_{n+1,1}, null, h_{n+1,2}, pk_{u_{n+1}}, (f'_1, f'_2, \dots, f'_{n+1}), Sig_{n+1}\}$ to all the members in the network.

4) Each member $u'_k (1 \leq k \leq n-1)$ received the messages $\{h_{n+1,1}, null, h_{n+1,2}, pk_{u_{n+1}}, (f'_1, f'_2, \dots, f'_{n+1}), Sig_{n+1}\}$ from u_{n+1} . They verify the identity of u_n by the equation

$e(Sig_{n+1}, g_1) = e(h_{n+1,2}, pk_{u_{n+1}})e(g_1 \sum_{r=1}^n f'_r, pk_{u_{n+1}})$. If it holds, u'_k can ensure the transaction sent by u_{n+1} . u'_k computes $\alpha'_k = m_k^{-1}f'_k, \beta'_k = \sum_{\lambda=1}^{n+1} f'_\lambda$ and the group key $gk'_{u_k} = e(\alpha'_k, \beta'_k)$.

4. Group key update for nodes exit group

When some terminals wish to leave the group, the remove phase occurs in this case. To protect the secure communication of the group, either create a new group key or modify the existing group key is necessary. In this section, we use the latter option to modify an existing group key so that no outgoing user can generate a group key in the future. Suppose the set of current group member is $U = \{u_1, u_2, \dots, u_n\}$, the corresponding identity set is $ID = \{id_{u_1}, id_{u_2}, \dots, id_{u_n}\}$, the terminal node $u_j (1 \leq j \leq n)$ wishes to exit this group, and the public/private key pair of the terminal u_j is (sk_{u_j}, pk_{u_j}) . The new set of terminal node is $U'' = U - \{u_j\} = \{u_1, u_2, \dots, u_{j-1}, u_{j+1}, \dots, u_n\}$. The updated steps of group key are as follows:

1) u_j computes $sig'_j = sk_{u_j} H_1(IP_j)g_1$. Then, it broadcasts the message sig'_j , its IP address and public key messages (IP_j, pk_{u_j}) as a transaction to the group in the network.

2) We suppose the last block in the blockchain is u_n . After receiving the messages from u_j , u_n verifies the identity of u_j by the equation $e(sig'_j, g_1) = e(H_1(IP_j)g_1, pk_{u_j})$. If it holds, u_n can ensure the transaction sent by block u_j . u_n will write a new block. u_n selects a random number $t'_n \in \mathbb{Z}_q^*$, calculates $M'_n = t'_n M_n, f''_1 = t'_n M_1, f''_2 = t'_n M_2, \dots, f''_{j-1} = t'_n M_{j-1}, f''_{j+1} = t'_n M_{j+1}, \dots, f''_n = t'_n M_n, h'_{n,2} = H_2(pk_{u_n}, M'_n, sk_{u_n})g_1$ and $Sig''_n = sk_{u_n} h'_{n,2} + sk_{u_n} M'_n$. Then, u_n encapsulates the messages $\{h_{n,1}, null, h'_{n,2}, pk_{u_n}, (f''_1, \dots, f''_{j-1}, f''_{j+1}, \dots, f''_n), Sig''_n\}$ into a block. u_n can calculate the group key $gk''_{u_n} = e(t'_n g_1, \sum_{r''=1, r'' \neq j}^n f_{r''})$, and broadcast the messages $\{h_{n,1}, null, h'_{n,2}, pk_{u_n}, (f''_1, \dots, f''_{j-1}, f''_{j+1}, \dots, f''_n), Sig''_n\}$ to all the members in the network.

3) Each member $u_k (1 \leq k < n, k \neq j)$ received the messages $\{h_{n,1}, null, h'_{n,2}, pk_{u_n}, (f''_1, \dots, f''_{j-1}, f''_{j+1}, \dots, f''_n), Sig''_n\}$ from u_n . They verify the identity of u_n by the equation $e(Sig''_n, g_1) = e(h'_{n,2}, pk_{u_n})e(M'_n, pk_{u_n})$. If it holds, u_k can ensure the transaction sent by u_n . u_k computes $\alpha''_k = m_k^{-1}f''_k, \beta''_k = \sum_{\lambda''=1, \lambda'' \neq j}^n f_{\lambda''}$ and the group key $gk''_{u_k} = e(\alpha''_k, \beta''_k)$.

IV. Protocol Evaluation

1. Security analysis

1) Security against passive adversary

In Step 1, An attacker can obtain the messages $\{h_{n,1}, null, h_{n,2}, pk_{u_n}, (f_1, f_2, \dots, f_n), Sig_n\}$ broadcast by

u_n over the blockchain, but the attacker who doesn't know m_i will be unable to compute α_i directly from f_i , because it is equal to the difficulty of ICDH problem (the security proof will be demonstrated in Lemma 1).

In Step 2, we will demonstrate that (f_i, M_i, α_i) and (f_i, M_i, π) are computationally indistinguishable, and π is a random value in G_1 and $\alpha_i = t_n g_1$ is key parameter of the established conference key) (the security proof will be demonstrated in Lemma 2). We know that obtaining the group key $gk_{u_i} = e(\alpha_i, \beta_i)$ from the parameters (f_i, M_i, π) and bilinear mapping e is equal to the difficulty of DBDH problem.

Lemma 1 Under the random oracle model, if computing the ICDH is hard, any malicious adversary $\mathcal{C}$ cannot compute the value α_i from f_i .

Proof $\mathcal{C}$ can receive the parameters $\{h_{n,1}, null, h_{n,2}, pk_{u_n}, (f_1, f_2, \dots, f_n), Sig_n\}$ from u_n , and it also obtains the value M_i from the transaction record of the block u_i . Since $f_i = t_n M_i = t_n m_i g_1$, $M_i = m_i g_1$ and $\alpha_i = t_n g_1$. If $\mathcal{C}$ wants to calculate the group key $gk_{u_i} = e(\alpha_i, \beta_i)$ without knowing m_i , it must compute α_i from f_i . We assume that $\mathcal{C}$ can compute the value α_i from f_i by an efficient algorithm $\mathcal{A}$. Then, $\mathcal{C}$ uses $f_i = abg_1$ and $M_i = ag_1$ as input, and outputs $\alpha_i = bg_1$. So it can use $\mathcal{A}$ to construct an efficient algorithm $\mathcal{A}'$ to solve the ICDH problem, which is a contradiction for the hard of ICDH problem.

Lemma 2 Under the decision bilinear Diffie-Hellman assumption, $(gk_{u_j}, M_i, \alpha_i)$ and (gk_{u_j}, M_i, π) are computationally indistinguishable, and π is a random value in G_1 .

Proof A passive attacker tries to learn secret information about the group key by listening to the communication channel. The passive attacker may obtain all pairs (M_j, f_j) , $1 \leq j \leq n$, where $M_j = m_j g_1$ and $f_j = t_n m_j g_1$. Here we show that the passive attacker cannot get any information about the group key $gk_{u_j} = e(t_n, \sum_{i=1}^n f_i)$. Assumption 1, we shall prove that $(gk_{u_j}, M_i, \alpha_i)$ and (gk_{u_j}, M_i, π) are computationally indistinguishable, and π is a random value in G_1 .

Using the contradiction proof, assume that there exists an algorithm $\mathcal{A}$, which can efficiently distinguish $(gk_{u_j}, M_i, \alpha_i)$ and (gk_{u_j}, M_i, π) . Based on the algorithm $\mathcal{A}$, we can construct another algorithm $\mathcal{A}'$ that can efficiently distinguish (y_1, y_2, bg_1) and (y_1, y_2, π) , where $y_1 = abg_1, y_2 = ag_1$ and $a, b \in \mathbb{Z}_q^*$. First, the values y_1, y_2 and π are the input of algorithm $\mathcal{A}'$. Without loss of generality, let $f_1 = y_1$ and $M_1 = y_2$. Then, Algorithm $\mathcal{A}'$ randomly selects $(\lambda_1, \lambda_2, \dots, \lambda_{n-1})$ from $\mathbb{Z}_q^*$ and calculates values are as follows:

$$M_2 = \lambda_1 g_1, f_2 = \lambda_1 \pi;$$

$$M_3 = \lambda_2 g_1, f_3 = \lambda_2 \pi;$$

$$\vdots$$

$$M_n = \lambda_{n-1} g_1, f_n = \lambda_{n-1} \pi$$

Therefore, the algorithm $\mathcal{A}'$ constructs all M_j, f_j , $1 \leq j \leq n$, and computes $e(\pi, \sum_{i=1}^n f_i) = e(\pi, abg_1 + (\lambda_1 + \dots + \lambda_{n-1})\pi)$, then $\mathcal{A}'$ calls $\mathcal{A}$ with these values. It is obvious that $bg_1 = \pi$ based on property of bilinear map if $e(\pi, \sum_{i=1}^n f_i) = e(bg_1, b(M_1 + M_2 + \dots M_n)) = e(bg_1, b \sum_{i=1}^n M_i) = e(bg_1, \sum_{i=1}^n f_i) = gk_{u_i}$ holds. $\mathcal{A}'$ can apply $\mathcal{A}$ to efficiently distinguish (abg_1, ag_1, bg_1) and (abg_1, ag_1, π) , which is a contradiction for the Decision Diffie-Hellman problem assumption. Thus, the proposed protocol is secure against passive attacks under the Decision Diffie-Hellman problem assumption.

Theorem 1 Under the random oracle model, the proposed protocol is secure against an impersonator's attack.

Proof By Lemma 1, we know that only one legal participant u_i with secret key sk_{u_i} can generate a valid $Sig_i = sk_{u_i} h_{i,2} + sk_{u_i} M_i$. Since an impersonator does not know any legal participant's secret key, the impersonator cannot compute the valid Sig_i . Therefore, the proposed protocol is secure against impersonation attack under the random oracle model.

We have demonstrated that if a malicious participant attempts to disrupt the establishment of a group key among honest participants, he will be detected by verifying the equation $e(Sig_i, g_1) = e(h_{i,2}, pk_{u_i})e(M_i, pk_{u_i})$. Through Lemmas 1 and 2, we will demonstrate that the proposed protocol is secure against attacks of active adversaries.

Theorem 2 Under the random oracle model, the proposed protocol is secure against attacks of active adversaries.

Proof Since each participant u_i follows the protocol, the group key gk_{u_i} is computed using (α_i, f_i) and $1 \leq i \leq n$. By Lemma 1, we know that only one honest participant u_i can generate a valid M_i . α_i is the form of $\alpha_i = m_i^{-1} f_i$ and computed by the honest participant u_i while knowing m_i , and the honest participant u_i has checked the correctness of group key by verifying the equation $e(\alpha_i, \sum_{i=1}^n M_i) = e(\beta_i, g_1)$. Therefore, the proposed protocol is secure against attacks of active adversaries under the random oracle model.

Theorem 3 Under the random oracle model, the proposed protocol is resistant to known-key attacks.

Proof Suppose that an adversary knows u_i with the group key gk_{u_i} attempts to learn other established group keys. Since gk_{u_i} does not include any long-

term information, and since each short-term secret m_i is randomly chosen by each participant at each conference, and as a result all established group keys are independent each other. To impersonate a participant in the next conference, the adversary must learn the participant's long-term secret key sk_{u_j} . For the same reason, since gk_{u_i} does not include any long-term information, the adversary is unable to derive any long-term information of the participants. If the adversary attempts to work out the participant's long-term sk_{u_j} from (pk_{u_j}, sig_j) , where $pk_{u_j} = sk_{u_j}g_1$, $Sig_j = sk_{u_j}h_{i,2} + sk_{u_i}M_i$, by Lemma 1, then we know that the adversary cannot learn any participant's long-term secret key sk_{u_j} . Therefore, the proposed protocol is resistant to any known-key attacks under the random oracle model.

2) Forward-security and backward-security

In order to prevent the new joining node from obtaining the group secret information before joining and the exiting node from accessing the group secret information after it drops out, the forward-security and backward-security are necessary.

When the block node has left the group u_i , the group key update will be occurring. That u_n randomly selects a number $t'_n \in \mathbb{Z}_q^*$ and computes the new group key is gk''_{u_n} . The left block attempts to learn the later group key gk''_{u_n} at an efficient Probabilistic polynomial time. Since updated group key is $gk''_{u_n} = e(t'_ng_1, \sum_{r''=1, r'' \neq j}^n f_r'')$ and it does not include f_i . The fact that the left block uses m_i to learn t'_ng_1 is clearly does not help him, because the left block must face the difficulty of computing the discrete logarithm problem when working out t'_ng_1 from $f_j'' = t'_nm_jg_1$.

Similarly, the block node u_{n+1} joins the group, the group key update will be occurring. That u_{n+1} randomly selects a number $t_{n+1} \in \mathbb{Z}_q^*$ and computes the new group key is $gk_{u_{n+1}}$. The joined block node learns the previous group key gk_{u_n} at an efficient Probabilistic polynomial time. Since $gk_{u_n} = e(t_ng_1, \sum_{r=1}^n f_r)$ and it does not include f_{n+1} . That the joined block cannot learn t_ng_1 , because it must face the difficulty of computing the discrete logarithm problem when working out t_ng_1 from $f_n = t_nm_ng_1$. Therefore, the proposed protocol possesses

forward and backward secrecy under the difficulty of computing the discrete logarithm problem.

2. Efficiency analysis

We compared and analyzed the literature that can be quantified in recent years. We compare the proposed protocol with the related works^[5,6,10,12] in communication costs, computation costs and time costs. Below are the few notations and data that are going to be used in comparison^[11]. The execution time of the encryption operation is calculated by MIRACL which is a well-known encryption library and has been widely used to implement encryption operations in many environments. Hardware platform consists of an Intel I7-4770 processor with 3.40 GHz clock frequency, 4 gigabytes memory and running Windows 7 operating system.

- T_{inv} : Execution time for calculating modular inverse operation, $T_{inv} \approx 0.174\text{ms}$.
- T_{mul} : Execution time for multiplication of two numbers, $T_{mul} \approx 0.015\text{ms}$.
- T_{exp} : Execution time for Exponentiation, $T_{exp} \approx 3.886\text{ms}$.
- T_{pa-ecc} : Execution time for elliptic curve point addition, $T_{pa-ecc} \approx 0.0018\text{ms}$.
- T_{sm-ecc} : Execution time for elliptic curve point multiplication, $T_{sm-ecc} \approx 0.442\text{ms}$.
- T_h : Execution time for hash operation, $T_h \approx 0.0001\text{ms}$.
- T_{bp} : Execution time for bilinear pairing operation, $T_{bp} \approx 4.211\text{ms}$.
- T_{mtp} : Execution time of map-to-point hash operation, $T_{mtp} \approx 4.406\text{ms}$.
- T_{pa-bp} : Execution time of the point addition related to bilinear pairing, $T_{pa-bp} \approx 0.071\text{ms}$.
- T_{sm-bp} : Execution time of the multiplication of scalar with the point based on bilinear pairing, $T_{sm-bp} \approx 1.709\text{ms}$.

Table 1 lists the comparisons and analysis between the BD-GKA protocol and other three group-key agreement protocols in the computational complexity, communication load and time cost.

From Table 1, BD-GKA protocol and Wang *et al.*^[12] have similar computational complexity and have the lowest computational complexity. Islam *et al.*^[6] and Teng *et al.*^[5] have higher computational complexity. Ermis *et al.*^[10] has the highest computational complexity.

Table 1. Complexity analysis of the four protocols

	Computational complexity of each ordinary node	The number of message sent	The number of message received
Ermis <i>et al.</i> ^[10]	$(8n-1)T_{exp} + 4nT_{mul} + (n+3)T_{inv} + 3T_{pa-ecc} + (n+2)T_h$	$9 q $	$(8n-8) q $
Islam <i>et al.</i> ^[6]	$(n-1)T_{bp} + (4n-3)T_{mul} + (5n-2)T_h$	$(3n-3) q $	$(3n-3) q $
Wang <i>et al.</i> ^[12]	$3T_{exp} + (2n-2)T_{sm-ecc} + 1T_{inv} + (n+3)T_h$	$12 q $	$(12n-12) q $
Teng <i>et al.</i> ^[5]	$(n-1)T_{bp} + (n-1)T_{exp} + 2nT_{sm-bp} + nT_{pa-ecc} + 3nT_h$	$(n-1) G + n q $	$(n-1) G + n q $
BD-GKA protocol	$9T_{bp} + 3T_{mul} + 1T_{inv} + (n+1)T_{pa-bp} + 4T_h$	$(n+4) G + 1 q $	$(2n+8) G + 3 q $

In terms of communication, Teng *et al.*^[5] and BD-GKA protocol have similar communication complexity and have the lowest communication complexity. Ermis *et al.*^[10] and Islam *et al.*^[6] have lower communication complexity. Wang *et al.*^[12] has the highest communication complexity.

For convenience and without loss of generality, we do time cost analysis for the ordinary nodes of group key agreement. Based on the data provided by Table 1, we compare the BD-GKA protocol with other four protocols in time cost. The result is show as Fig.2.

From Fig.2, time cost of the ordinary nodes in the five group key agreement protocols, BD-GKA protocol and Wang *et al.*^[12] have similar time and the lowest time cost of ordinary nodes, followed by Islam *et al.*^[6] and Teng *et al.*^[5] protocol. Ermis *et al.*^[10] has the highest time cost.

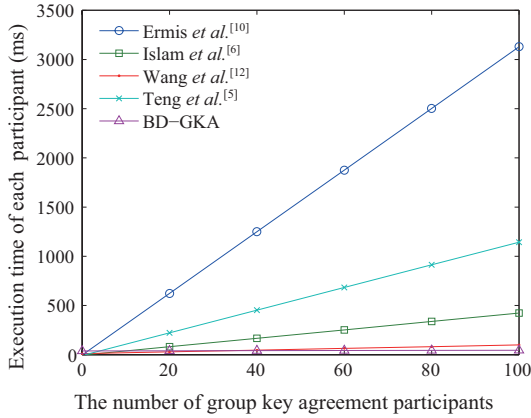


Fig. 2. The execution time of the five protocols

We analyze the computational energy consumption of the five group key agreement protocols using the data provided in Makri *et al.*^[3]. For convenience, the consumption energy cost of these protocols is simply the sum of the modular exponentiation computation, scalar multiplication and Tate pairing computation cost, and a 133MHz “Strong ARM” microprocessor consumes 9.1mJ for performing a modular exponentiation, 8.8mJ for performing a scalar multiplication, 47.0mJ for a Tate pairing, 8.8mJ for performing a Elliptic Curve Digital Signature Algorithm and 10.9mJ for performing a Elliptic Curve Digital Signature Verify Algorithm. The computation energy consumption of five group key agreement protocols is shown in Fig.3.

The computational energy consumption of the examined Protocols is depicted in Fig.3. Ermis *et al.*^[10] has the worst performance followed by Islam *et al.*^[6] and Teng *et al.*^[5] protocol, the BD-GKA protocol is very effective in terms of computational energy consumption.

As for the communication energy cost provided in Makri *et al.*^[3], an IEEE 802.11 Spectrum 24 WLAN card consumes 0.00066mJ for the transmission of 1bit and

0.00031mJ for the reception of 1bit. The above mentioned energy costs will be used for the communication energy analysis of the five GKA protocols. To compute the communication cost, we need to know the size of the information exchange. We assume that the secure key length is 160b of the elliptic curve cryptography and the information exchange length used by all five protocols is 160b, and the communication consumption is as shown in Fig.4.

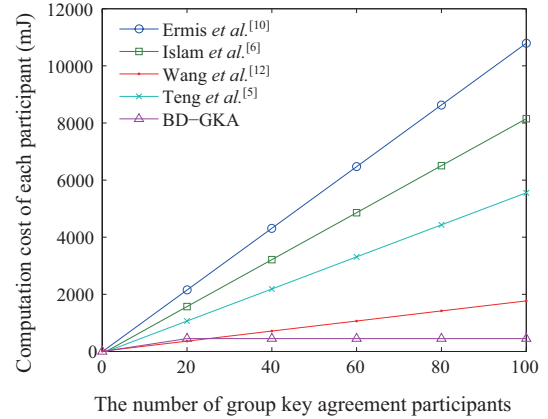


Fig. 3. The computation cost of the five protocols

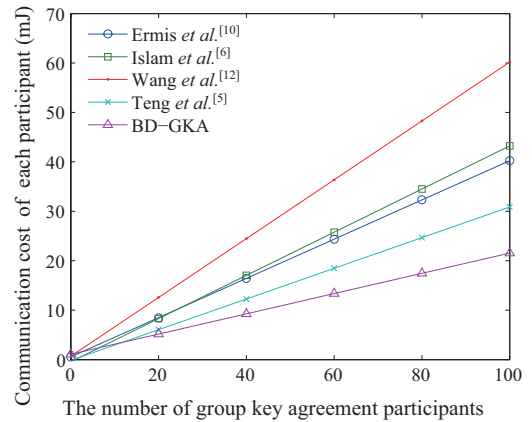


Fig. 4. The communication cost of the five protocols

The communication consumption of the examined Protocols is depicted in Fig.4, the communication energy consumption increases rapidly as the nodes increased in Wang *et al.*^[12], the cost of Ermis *et al.*^[10] and Islam *et al.*^[6] is similar in terms of communication consumption, and BD-GKA has the least communication power consumption.

VI. Conclusion

This paper analyzed the application requirements of group key agreement in the mobile ad hoc network and the problems that need to be solved of group key negotiation in this network environment, and proposed a blockchain-based dynamic group key agreement protocol

for ad hoc network. It used the technical advantages of blockchain to achieve a non-centralized authentication, lightweight calculation key self-certified and dynamic group key agreement protocol. It solves the technical bottleneck of group key agreement in ad hoc network, and satisfies the application requirements of key information exchange in this network environment.

References

- [1] W. Diffie and M. Hellman, "New directions in cryptography", *IEEE transactions on Information Theory*, Vol.22, No.6, pp.644–654, 1976.
- [2] Q. Zhang, Y. Li, Y. Gan, *et al.*, "Group key agreement protocol based on privacy protection and attribute authentication", *IEEE Access*, Vol.7, pp.87085–87096, 2019.
- [3] E. Makri and E. Konstantinou, "Constant round group key agreement protocols: A comparative study", *Computers and Security*, Vol.30, No.8, pp.643–678, 2011.
- [4] Q. Zhang, J. Yuan, G. Guo, *et al.*, "An authentication key establish protocol for WSNs based on combined key", *Wireless Personal Communications*, Vol.99, No.1, pp.95–110, 2018.
- [5] J. Teng and C. Wu, "A provable authenticated certificateless group key agreement with constant rounds", *Journal of Communications and Networks*, Vol.14, No.1, pp.104–110, 2012.
- [6] S. Islam and A. Singh, "Provably secure one-round certificateless authenticated group key agreement protocol for secure communications", *Wireless Personal Communications*, Vol.85, No.3, pp.879–898, 2015.
- [7] Q. Zhang, Q. Zhang, Z. Ma, *et al.*, "An authenticated asymmetric group key agreement for imbalanced mobile networks", *Chinese Journal of Electronics*, Vol.23, No.4, pp.827–835, 2014.
- [8] Q. Zhang, Y. Li, Z. Li, *et al.*, "Access control based on ciphertext attribute authentication and threshold policy for the internet of things", *Sensors*, Vol.19, No.23, Article ID 5237, 12 pages, 2019.
- [9] J. Zheng, X. Zhang, Q. Zhang, *et al.*, "Multi-domain lightweight asymmetric group key agreement", *Chinese Journal of Electronics*, Vol.27, No.5, pp.1085–1091, 2018.
- [10] O. Ermis, S. Bahtiyar, E. Anarim, *et al.*, "A secure and efficient group key agreement approach for mobile ad hoc networks", *Ad Hoc Networks*, Vol.67, pp.24–39, 2017.
- [11] S. Islam, M. Obaidat, P. Vijayakumar, *et al.*, "A robust and efficient password-based conditional privacy preserving authentication and group-key agreement protocol for VANETs", *Future Generation Computer Systems*, Vol.84, pp.216–227, 2018.
- [12] M. Wang and Z. Yan, "Privacy-preserving authentication and key agreement protocols for D2D group communications", *IEEE Transactions on Industrial Informatics*, Vol.14, No.8, pp.3637–3647, 2018.
- [13] Y. Tan, J. Zheng, Q. Zhang, *et al.*, "A Specific-targeting asymmetric group key agreement for cloud computing", *Chinese Journal of Electronics*, Vol.27, No.4, pp.866–872, 2018.
- [14] J. Zheng, Y. Tan, Q. Zhang, *et al.*, "Cross-cluster asymmetric group key agreement for wireless sensor networks", *Science China Information Sciences*, Vol.61, No.4, Article ID 048103, 3 pages, 2018.
- [15] Q. Zhang, Y. Gan, Q. Zhang, *et al.*, "A dynamic and cross-domain authentication asymmetric group key agreement in telemedicine application", *IEEE Access*, Vol.6, pp.24064–24074, 2018.
- [16] P. Vijayakumar, R. Naresh, L. Jegatha Deborah, *et al.*, "An efficient group key agreement protocol for secure P2P communication", *Security and Communication Networks*, Vol.9, No.17, pp.3952–3965, 2016.
- [17] Q. Zhang, Y. Li, Q. Zhang, *et al.*, "A self-certified cross-cluster asymmetric group key agreement for wireless sensor networks", *Chinese Journal of Electronics*, Vol.28, No.2, pp.280–287, 2019.



ZHANG Qikun was born in 1980.03. received his Ph.D. from Beijing Institute of Technology in 2013. Now he is a associate professor in department of Computer and Communication Engineering, Zhengzhou University of Light Industry, Zhengzhou, China. His research interests include information security and cryptography (Email:zhangqikun04@163.com)